# TP 8 : EXCEPTIONS

**Langage C++**

**Consigne :** Le TP sera déposé sur Moodle au plus tard une semaine après la séance.

Dans ce TP, vous allez implémenter un système de manipulation de données tabulaires (DataFrame) avec la prise en charge d'exceptions.

## I. Définition des exceptions

Afin de gérer les erreurs susceptibles de se produire lors de l'utilisation du dataframe, nous allons créer plusieurs exceptions personnalisées dérivant toutes de la classe de base `DataFrameException`.
Dans le reste du TP vous n'utiliserez que les exceptions `ColumNotFoundException`, `IndexOutOfBoundsException`, `TypeMismatchException`, et `DataFrameException`.

1. Créer la classe d'exception `ColumnNotFoundException` qui est utilisée lorsqu'on essaie d'accéder ou d'utiliser une colonne inexistante ;
2. Créer la classe d'exception `IndexOutOfBoundsException` qui est utilisée lorsqu'on essaie d'accéder à un index au-delà d'une limite définie ;

## II. Classe `Value`

La classe `Value` permet de représenter une valeur dans le dataframe. Les types de valeur supportés sont identifiés par des entiers à l'aide d'attributs statiques.

```
static const int TYPE_NONE = 0;
static const int TYPE_INTEGER = 1;
static const int TYPE_DOUBLE = 2;
static const int TYPE_STRING = 3;
```

Le champ `m_type` contient l'identifiant du type de la valeur contenue. Les attributs `m_intValue`, `m_doubleValue` et `m_stringValue` permettent de stocker la valeur représentée, ils ne peuvent pas être utilisé simultanément.

Les accesseurs `asInt()`, `asDouble()` et `asString()` retournent la valeur contenue en fonction du type. Chaque accesseur essaie de convertir la valeur contenue vers le type de retour, si la conversion n'est pas possible une exception de type `TypeMismatchException` est émise.

## III. Classe `Column`

La classe `Column` permet de représenter une colonne du dataframe pouvant contenir plusieurs valeurs de même type. Implémentez les méthodes suivantes :

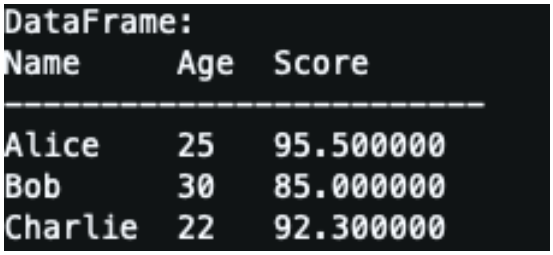
1. Le constructeur de la classe ;
2. `getName()` qui retourne le nom de la colonne ;
3. `getType()` qui retourne le type des valeurs de la colonne ;
4. `size()` qui retourne la taille actuelle de la colonne ;
5. `at(size_t index)` qui retourne la valeur à la position `index`, si l'index n'existe pas une exception est émise ;
6. `addValue(const Value& value)` qui ajoute une valeur à la colonne, si le type de `value` est différent de celui de la colonne, une exception est émise ;
7. `isNumeric()` qui indique si le type des valeurs de la colonne est numérique ;
8. `mean()` qui calcule et retourne la moyenne des valeurs, attention cela n'est possible que si le type de la colonne est numérique et qu'elle n'est pas vide dans le cas contraire une exception est émise ;

## IV. Classe `DataFrame`

La classe `DataFrame` contient plusieurs colonnes, chaque colonne peut avoir un type différent. Le nombre de lignes doit être le même pour chaque colonne. Implémentez les méthodes suivantes :

1. Le constructeur de la classe ;
2. `addColumn(const std::string& name, int type)` qui permet de créer une nouvelle colonne et de l'ajouter au dataframe, si le nom de la colonne existe déjà une exception est émise. Si la colonne est ajoutée alors que le nombre de lignes du dataframe est supérieur à 0, des valeurs (`Value()`) vides sont insérées dans la colonne afin de l'équilibrer ;
3. `hasColumn(const std::string& name)` qui vérifie qu'une colonne existe ;
4. `getColumn(const std::string& name)` qui retourne une colonne et émet une exception si elle n'existe pas ;
5. `rowCount()` retourne le nombre de lignes du dataframe ;
6. `columnCount()` retourne le nombre de colonne du dataframe ;
7. `getColumnNames()` retourne le nom des colonnes ;
8. `addrow(const std::map<std::string, Value>& values)` qui ajoute une ligne au dataframe, la ligne contient une valeur par colonne d'où l'utilisation d'une map. Si une colonne n'existe pas, une exception est émise. SI une value ne correspond pas au type de sa colonne une exception est émise ;
9. `getValue(const std::string& columnName, size_t rowIndex)` retourne une cellule du dataframe (la valeur à la position `rowIndex` de la colonne de nom `columnName`). Si l'index dépasse la limite de la colonne, une exception est émise ;

10. `setValue(const std::string& columnName, size_t rowIndex, const Value& value)` met à jour la valeur d'une cellule. SI le type de value ne correspond pas à celui de la colonne cible, une exception est émise. Si l'index dépasse la limite de la colonne, une exception est émise ;
11. `print(std::ostream& os=std::cout, size_t maxRows=10)` affiche le contenu du dataframe dans un flux de sortie (par défaut sur la sortie standard). L'argument `maxRows` définit le nombre maximal de ligne à afficher.




## V. Test

La fonction main() ne doit pas être modifiée, elle est utilisée pour tester votre programme et vérifie que les exceptions attendues sont bien émises.

```
DataFrame:
Name     Age  Score
-----------------------
Alice    25   95.500000
Bob      30   85.000000
Charlie  22   92.300000
Exception caught: Column not found: GPA
Exception caught: Index 10 out of bounds (max: 2)
Exception caught: Type mismatch: expected INTEGER, got STRING
Mean Age: 25.6667
Mean Score: 90.9333
Exception caught: Cannot calculate mean on non-numeric column
Mean of Score: 90.9333
Exception caught in outer try block: Error in statistics calculation: Cannot calculate mean on non-numeric column
```